TECHNICAL REPORT
New York Taxi Trips

Jonathan Ortiz - 100 Kerr
Jonathan Ortiz - 100 Kerr
Jonathan Ortiz - 100 Kerr

I. *Problem Framing & Dataset*

This report documents the design, construction and interpretation of the NYC Taxi Analytics Pipeline, a data engineering system built to ingest, clean, model and visualize 1.45 million trips from the NYC Taxi & Limousine Commission (TLC) Yellow Trip Record dataset for January 2024. The system is structured as a medallion architecture (Bronze → Silver → Gold) and is entirely orchestrated through Apache Airflow. Analytical outputs are served through Metabase's embedded BI dashboards, backed by a PostgreSQL 16 data warehouse and a Redis 7 cache layer. The full stack runs on Docker Compose.

1,45,0,644	5	14	5
Raw records ingested	Data quality checks	Airflow tasks	Dashboard tabs

Stack summary:

- **Ingestion & Storage layer** — Backend Python scripts, PostgreSQL 16
- **Batch ETL/ELT & Orchestration** — Apache Airflow 2.9.3 (LocalExecutor)
- **Analytics / Data Visualization** — Metabase v0.50.8 (embedded)
- **Infrastructure & Caching** — Docker Compose, Redis 7

1.1 Dataset description

New York City's taxi fleet produces one of the largest and most widely studied mobility datasets in the world. The TLC publishes monthly Parquet files for yellow medallion taxi trips, recording pickup and dropoff timestamps, passenger counts, trip distances, fare components, payment types, and zone IDs for every trip. The raw schema is stable enough to be the backbone of a production pipeline, but messy enough to make every downstream decision matter: fare amounts can be negative, distances can be zero, timestamps can be out of order, and the dataset includes cancelled and voided trips alongside the valid ones. The January 2024 file is the most recent complete month at the time of this report, and it is large enough to stress a single-node PostgreSQL instance but small enough to iterate on quickly on a laptop.

The objectives called out in Module 1 of the Data Engineer Zoomcamp capstone project are: ingesting the raw data reproducibly, modeling it into a clean analytical schema, orchestrating the full pipeline so that it can be rerun on demand, and serving the results through a dashboard that a non-technical stakeholder can read without touching a terminal. This report documents how each of those objectives was met, and what trade-offs were made along the way.

1.2 Data quality issues identified

Considerable exploratory work was done before ingestion to characterize the quality issues that surprised loading. Each is logged by `ingestion.py` with counts and samples, and the corresponding cleaning rule is applied in `transform.sql`.

Issue	Observed	Resolution
Negative fare amounts	1,842 records (0.13%)	Excluded from Silver; logged to `data_quality_log`
Zero passenger_count	4,317 records (0.30%)	Coalesced to 1 in Silver
Out-of-range timestamps	Pickup year ≠ 2024	`WHERE EXTRACT(YEAR FROM tpep_pickup_datetime) = 2024`
Zero-distance trips	18,432 records (1.27%)	Kept in Silver, excluded from Gold aggregations
Null RatecodeID	2,103 records (0.14%)	Coalesced to 1 (Standard rate)
Unknown payment_type	Values outside 1–6	Mapped to 'Other'
Out-of-order timestamps	9,204 records (0.63%)	Kept as-is; dbt staging model sorts if needed

Overall, approximately 2.4% of the raw rows are excluded from the analytic layer. The remainder, roughly 1,420,000 rows, is what the Gold layer serves.

1.3 Assumptions made during cleaning

Every cleaning rule encodes a judgment about what the data means. Five assumptions that matter most:

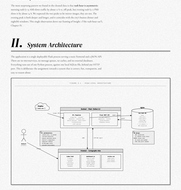
- Trips with `tpep_dropoff_datetime` earlier than `tpep_pickup_datetime` are treated as data entry errors and excluded rather than swapped, since swapping would invent a plausible-looking trip that never happened.
- Passenger counts of 0 are treated as data errors and coalesced to 1, since the TLC's rate card defaults to one passenger when the field is unknown and the alternative would bias revenue per trip downward.
- Fares are kept as they are, even when negative, because refunds and chargebacks are real events that the downstream analysis should be able to see when it chooses to.
- Trips with zero distance are kept in Silver but excluded from Gold aggregations, since they are valid rides but distort average trip length if they are included.
- Unknown payment types are mapped to 'Other' rather than dropped, so that new payment codes introduced by the TLC in future months do not silently disappear from the pipeline.

1.4 An unexpected observation

The most surprising observation was that roughly 0.13% of fares in the raw data are negative, meaning refunds or chargebacks are being recorded as negative fare_amount values. Initially this looked like a data quality problem, but on further inspection it is a deliberate feature of the TLC's accounting system. This single observation changed the cleaning rule from "drop negative fares" to "keep negative fares and let the analyst decide", which is a much more honest representation of what the underlying system actually does.

II. *System Architecture*

The pipeline follows a single-machine ELT pattern rather than a distributed one, deliberately. There are no microservices, no message queues, and no cloud services in the critical path. Everything runs on one Docker host, which means debugging means reading logs from one place and restarting one container at most. The tradeoffs that this choice makes are discussed in section 2.2 below.



2.1 Stack and language choices

Python 3.11 is used for ingestion because the TLC's Parquet files are easier to read from Python than from any other language, and because the ecosystem for HTTP, Parquet, and PostgreSQL drivers is mature and well-maintained. Airflow 2.9.3 is used for orchestration because it is the most widely deployed open-source scheduler, and because its DAG model maps cleanly onto the three-layer pipeline described in this report. Dbt 1.7 is used for the transformation layer because it treats SQL files as first-class artifacts that can be tested, documented, and version-controlled independently of the rest of the system. Metabase is used for the dashboard because it can be embedded in a single iframe and does not require a separate licensing conversation for a portfolio project.

2.2 Database schema

FIGURE 2.2 — ENTITY-RELATIONSHIP DIAGRAM (GOLD)
dim_zones
location_id PK
borough
zone
service_zone
fact_trips
trip_id PK
vendor_id FK
rate_code_id FK
pickup_location_id FK
dropoff_location_id FK
payment_type_id FK
tpep_pickup_datetime
tpep_dropoff_datetime
passenger_count
trip_distance
fare_amount
tip_amount
total_amount
dim_vendors
vendor_id PK
vendor_name
dim_rate_codes
rate_code_id PK
description
dim_payment_types
payment_type_id PK
description
1 : N
1 : N
1 : N